

Multi-Layer Perceptrons and Deep Learning

Christos Dimitrakakis

June 27, 2026

Outline

Features and layers

- Introduction

- Layers

- Activation functions

Algorithms

- Random projection

- Back propagation

- Derivatives

- Cost functions

- Stochastic gradient descent in practice

Python libraries

- sklearn

- PyTorch

- TensorFlow

Features and layers

- Introduction

- Layers

- Activation functions

Algorithms

- Random projection

- Back propagation

- Derivatives

- Cost functions

- Stochastic gradient descent in practice

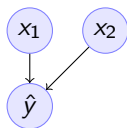
Python libraries

- sklearn

- PyTorch

- TensorFlow

Perceptron vs linear regression



- Network output

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2$$

- Chain rule

$$\nabla_{\beta} L = \nabla_{\hat{y}} L \nabla_{\beta} \hat{y}$$

- Network gradient

$$\nabla_{\beta} \hat{y} = (x_1, x_2)$$

Cost functions

The only difference are the cost functions

- Perceptron

$$L = -\mathbb{I}\{y \neq \hat{y}\} \hat{y}$$

with

$$\nabla L = -\mathbb{I}\{y \neq \hat{y}\} yx$$

- Linear regression

$$L = (\hat{y} - y)^2,$$

with

$$\nabla_{\hat{y}} L = 2(\hat{y} - y).$$

Layering and features

Fixed layers

- ▶ Input to layer $x \in R^n$
- ▶ Output from layer $\hat{y} \in R^m$.

Intermediate layers

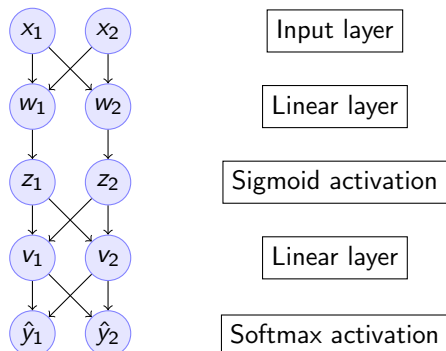
- ▶ Linear layer
- ▶ Non-linear **activation** function.

Linear layers types

- ▶ Dense
- ▶ Sparse
- ▶ Convolutional

Activation function

- ▶ Sigmoid
- ▶ Softmax



Linear layers

Example: Linear regression with n inputs, m outputs.

- ▶ Input: Features $\mathbf{x} \in \mathbb{R}^n$
- ▶ Dense linear layer with $\mathbf{B} \in \mathbb{R}^{m \times n}$
- ▶ Output: $\hat{\mathbf{y}} \in \mathbb{R}^m$

Dense linear layer

- ▶ Parameters $\mathbf{B} = \begin{pmatrix} \beta_1 \\ \vdots \\ \beta_m \end{pmatrix}$,
- ▶ $\beta_i = [\beta_{i,1}, \dots, \beta_{i,n}]$, β_i connects the i -th output y_i to the features $\mathbf{x}$:
$$y_i = \beta_i \mathbf{x}$$
- ▶ In compact form:

$$\mathbf{y} = \mathbf{B}\mathbf{x}$$

Multiple linear layers

Repeated linear transformations are linear

It does not really help to have multiple linear layers one after the other. For example, if we transform $x \in \mathbb{R}^n$ to $z \in \mathbb{R}^k$ to $y \in \mathbb{R}^m$ through two matrices

$$z = Ax, \quad A \in \mathbb{R}^{k \times n} \quad (1)$$

$$y = Bz, \quad B \in \mathbb{R}^{m \times k} \quad (2)$$

We can rewrite y as

$$y = B(Ax) = (BA)x = Cx, \quad C \in \mathbb{R}^{m \times n} \quad (3)$$

where $C = BA$.

- ▶ Successive linear layers have no advantage normally.¹
- ▶ However, we can interlace them with **non-linear activation functions**.

¹Multi-task learning might be an exception.

ReLU activation

- ▶ Activation function:

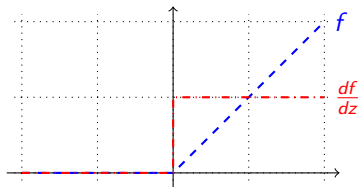
$$f(x) = \max(0, x)$$

- ▶ Derivative

$$\frac{d}{dx}f(x) = \mathbb{I}\{x > 0\}$$

Typically used in the hidden layers of neural networks, as it is:

- ▶ Simple to calculate.
- ▶ Nonlinear.
- ▶ Its gradient never vanishes.



Sigmoid activation

Example: Logistic regression

- ▶ Input $\mathbf{x} \in \mathbb{R}^n$
- ▶ Intermediate output: $z \in \mathbb{R}$,

$$z = \sum_{i=1}^n \beta_i x_i.$$

- ▶ Output: sigmoid activation $\hat{y} \in [0, 1]$.

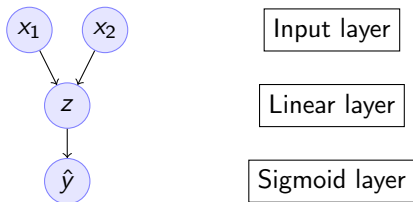
$$f(z) = 1/[1 + \exp(-z)].$$

Now we can interpret

$$\hat{y} = P_{\beta}(y = 1 | \mathbf{x}).$$

Loss function: negative log likelihood

$$\ell(\hat{y}, y) = -[\mathbb{I}\{y = 1\} \ln(\hat{y}) + \mathbb{I}\{y = -1\} \ln(1 - \hat{y})]$$



Softmax layer

Example: Multivariate logistic regression with m classes.

► Input: **Features** $\mathbf{x} \in \mathbb{R}^n$

► Fully-connected **linear** activation layer

$$\mathbf{z} = \mathbf{B}\mathbf{x}, \quad \mathbf{B} \in \mathbb{R}^{m \times n}.$$

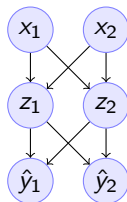
► **Softmax** output

$$\hat{y}_i = \frac{\exp(z_i)}{\sum_{j=1}^m \exp(z_j)}$$

We can also interpret this as

$$\hat{y}_i \triangleq \mathbb{P}(y = i \mid \mathbf{x})$$

with usual loss $\ell(\hat{y}, y) = -\ln \hat{y}_y$



Input layer

Linear layer

Softmax layer

Features and layers

Introduction

Layers

Activation functions

Algorithms

Random projection

Back propagation

Derivatives

Cost functions

Stochastic gradient descent in practice

Python libraries

sklearn

PyTorch

TensorFlow

Random projections

- ▶ Features x
- ▶ Hidden layer activation z
- ▶ Output y

Hidden layer: Random projection

Here we project the input into a high-dimensional space

$$z_i = \text{sgn}(\beta_i^\top x) = y_i$$

where $B = [\beta_i]_{i=1}^m$, $\beta_{i,j} \sim \text{Normal}(0, 1)$

The reason for random projections

- ▶ The high dimension makes it easier to learn.
- ▶ The randomness ensures we are not learning something spurious.

Background on back-propagation

Gradient descent algorithm

- ▶ We need to minimise the expected value $\mathbb{E}_{\beta}[L]$ of the loss function L
- ▶ Since we cannot calculate $\mathbb{E}_{\beta}[L]$, we use:

$$\nabla_{\beta} \mathbb{E}_{\beta}[L] \approx \frac{1}{T} \sum_{t=1}^T \nabla_{\beta} \ell(x_t, y_t, \beta).$$

- ▶ We can then update our parameters to reduce the **empirical loss**

$$\beta_{t+1} = \beta_t - \alpha_t \nabla_{\beta} \ell(x_t, y_t, \beta).$$

The problem

- ▶ However ℓ is a complex function of β .
- ▶ How can we obtain $\nabla_{\beta} \ell$?

The solution

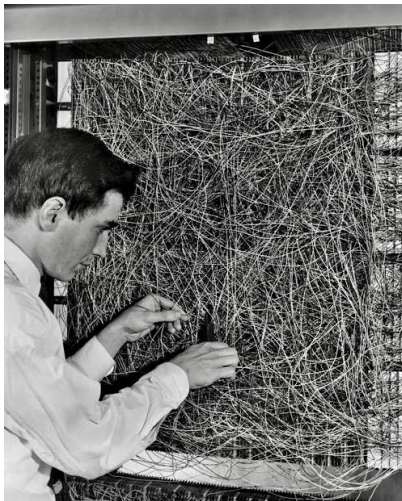
- ▶ Use the chain rule to "backpropagate" errors.

The chain rule of differentiation



[1673] Leibniz

Chain rule applied to the perceptron



[1976] Rosenblat

Chain rule for deep neural networks



[1982] Werbos

Backpropagation given a name

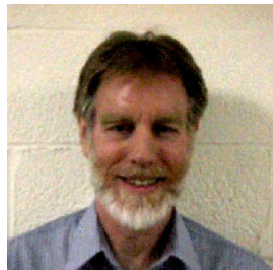
1986: Learning representations by back-propagating errors.



Rumelhart

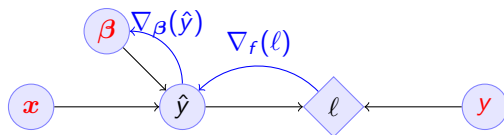


Hinton



Williams

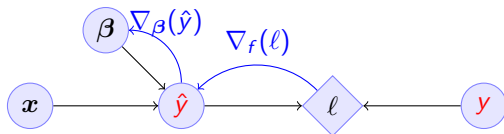
Elementary back-propagation: linear regression



- $f : X \rightarrow Y$, $\ell : Y \times Y \rightarrow \mathbb{R}$, chain rule: $\nabla_{\beta} \ell = \nabla_{\beta} f \nabla_{\hat{y}} \ell$
- **Forward**: follow the arrows to calculate **variables**

$$\hat{y} \triangleq f(\beta, x) = \sum_{i=1}^n \beta_i x_i, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

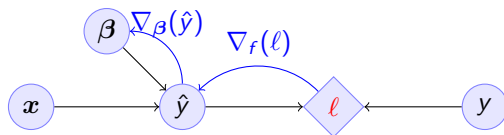
Elementary back-propagation: linear regression



- ▶ $f : X \rightarrow Y$, $\ell : Y \times Y \rightarrow \mathbb{R}$, chain rule: $\nabla_\beta \ell = \nabla_\beta f \nabla_{\hat{y}} \ell$
- ▶ **Forward**: follow the arrows to calculate **variables**

$$\hat{y} \triangleq f(\beta, x) = \sum_{i=1}^n \beta_i x_i, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

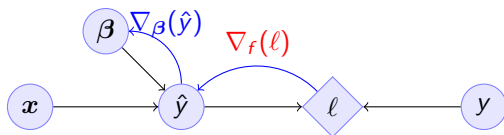
Elementary back-propagation: linear regression



- ▶ $f : X \rightarrow Y$, $\ell : Y \times Y \rightarrow \mathbb{R}$, chain rule: $\nabla_\beta \ell = \nabla_\beta f \nabla_{\hat{y}} \ell$
- ▶ **Forward**: follow the arrows to calculate **variables**

$$\hat{y} \triangleq f(\beta, x) = \sum_{i=1}^n \beta_i x_i, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

Elementary back-propagation: linear regression



- ▶ $f : X \rightarrow Y$, $\ell : Y \times Y \rightarrow \mathbb{R}$, chain rule: $\nabla_{\beta} \ell = \nabla_{\beta} f \nabla_{\hat{y}} \ell$
- ▶ **Forward:** follow the arrows to calculate **variables**

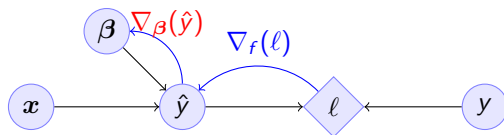
$$\hat{y} \triangleq f(\beta, x) = \sum_{i=1}^n \beta_i x_i, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

- ▶ **Backward:** return to calculate the **gradients**

$$\nabla_{\beta} \ell(\hat{y}, y) = \nabla_{\beta} f(\beta, x) \times \nabla_{\hat{y}} \ell(\hat{y}, y) \quad (4)$$

$$= \nabla_{\beta} f(\beta, x) \times 2[\hat{y} - y] \quad (5)$$

Elementary back-propagation: linear regression



- ▶ $f : X \rightarrow Y$, $\ell : Y \times Y \rightarrow \mathbb{R}$, chain rule: $\nabla_{\beta} \ell = \nabla_{\beta} f \nabla_{\hat{y}} \ell$
- ▶ **Forward:** follow the arrows to calculate **variables**

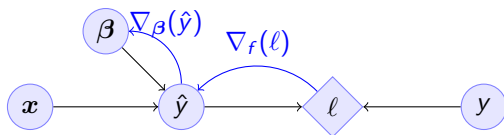
$$\hat{y} \triangleq f(\beta, x) = \sum_{i=1}^n \beta_i x_i, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

- ▶ **Backward:** return to calculate the **gradients**

$$\nabla_{\beta} \ell(\hat{y}, y) = \nabla_{\beta} f(\beta, x) \times \nabla_{\hat{y}} \ell(\hat{y}, y) \quad (4)$$

$$= \nabla_{\beta} f(\beta, x) \times 2[\hat{y} - y] \quad (5)$$

Elementary back-propagation: linear regression



► $f : X \rightarrow Y$, $\ell : Y \times Y \rightarrow \mathbb{R}$, chain rule: $\nabla_{\beta} \ell = \nabla_{\beta} f \nabla_{\hat{y}} \ell$

► **Forward:** follow the arrows to calculate **variables**

$$\hat{y} \triangleq f(\beta, x) = \sum_{i=1}^n \beta_i x_i, \quad \ell(\hat{y}, y) = (\hat{y} - y)^2$$

► **Backward:** return to calculate the **gradients**

$$\nabla_{\beta} \ell(\hat{y}, y) = \nabla_{\beta} f(\beta, x) \times \nabla_{\hat{y}} \ell(\hat{y}, y) \quad (4)$$

$$= \nabla_{\beta} f(\beta, x) \times 2[\hat{y} - y] \quad (5)$$

► Update:

$$\beta_{t+1} = \beta_t - \alpha_t \times \nabla_{\beta} \ell(\hat{y}_t, y_t)$$

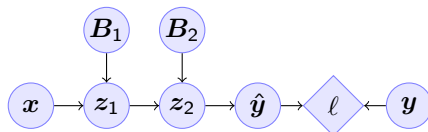
Gradient descent with *back-propagation*

- ▶ Dataset D , cost function $L = \sum_t \ell_t$
- ▶ Parameters $B_1, \dots, B_k$ with k layers
- ▶ Intermediate variables: $z_j = h_j(z_{j-1}, B_j)$, $z_0 = x$, $z_k = \hat{y}$.

Gradient descent with *back-propagation*

- ▶ Dataset D , cost function $L = \sum_t \ell_t$
- ▶ Parameters $B_1, \dots, B_k$ with k layers
- ▶ Intermediate variables: $z_j = h_j(z_{j-1}, B_j)$, $z_0 = x$, $z_k = \hat{y}$.

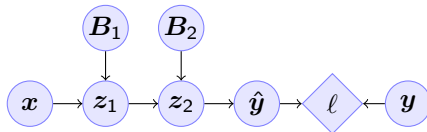
Dependency graph



Gradient descent with *back-propagation*

- ▶ Dataset D , cost function $L = \sum_t \ell_t$
- ▶ Parameters $B_1, \dots, B_k$ with k layers
- ▶ Intermediate variables: $z_j = h_j(z_{j-1}, B_j)$, $z_0 = x$, $z_k = \hat{y}$.

Dependency graph



Backpropagation with steepest stochastic gradient descent

- ▶ Forward step: For $j = 1, \dots, k$, calculate $z_j = h_j(k)$ and $\ell(\hat{y}, y)$
- ▶ Backward step: Calculate $\nabla_{\hat{y}} \ell$ and $d_j \triangleq \nabla_{B_j} \ell = \nabla_{B_j} z_j d_{j+1}$ for $j = k \dots, 1$
- ▶ Apply gradient: $B_j \leftarrow B_j - \alpha d_j$.

Other algorithms and gradients

Natural gradient

Defined for probabilistic models

ADAM

Exponential moving average of gradient and square gradients

BFGS: Broyden–Fletcher–Goldfarb–Shanno algorithm

Newton-like method

Linear layer

Definition

This is a linear combination of inputs $x \in \mathbb{R}^n$ and parameter matrix

$$B \in \mathbb{R}^{m \times n} \text{ where } B = \begin{bmatrix} \beta_1 \\ \vdots \\ \beta_i \\ \vdots \\ \beta_m \end{bmatrix} = \begin{bmatrix} \beta_{1,1} & \cdots & \beta_{1,j} & \cdots & \beta_{1,n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \beta_{i,1} & \cdots & \beta_{i,j} & \cdots & \beta_{i,n} \\ \vdots & \ddots & \vdots & \ddots & \vdots \\ \beta_{m,1} & \cdots & \beta_{m,j} & \cdots & \beta_{m,n} \end{bmatrix}$$

$$f(B, x) = Bx \quad f_i(B, x) = \beta_i \cdot x = \sum_{j=1}^n \beta_{i,j} x_j,$$

Gradient

Each partial derivative is simple:

$$\frac{\partial}{\partial \beta_{i,j}} f_k(B, x) = \sum_{k=1}^n \frac{\partial}{\partial \beta_{i,j}} \beta_{i,k} x_k = x_j$$

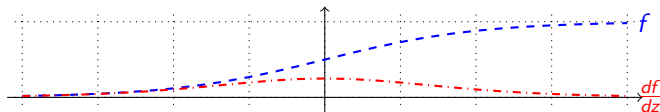
Sigmoid layer

- ▶ This layer is used for **binary classification**.
- ▶ It is used in the **logistic regression** model to obtain label probabilities.

$$f(z) = 1/(1 + \exp(-z))$$

- ▶ Derivative

$$\frac{d}{dz} f(z) = \exp(-z)/[1 + \exp(-z)]^2$$



Softmax layer

- ▶ This layer is used for **multi-class classification**
- ▶ It is a straightforward generalisation of the sigmoid function.

$$y_i(z) = \frac{\exp(z_i)}{\sum_j \exp(z_j)}$$

Derivative

$$\frac{\partial}{\partial z_i} y_i(z) = \frac{e^{z_i} e^{\sum_{j \neq i} z_j}}{\left(\sum_j e^{z_j}\right)^2}$$

$$\frac{\partial}{\partial z_i} y_k(z) = \frac{e^{z_i + z_k}}{\left(\sum_j e^{z_j}\right)^2}$$

Classification cost functions

Classification error

If z is the output for each class then

$$\ell(z, y) = \mathbb{I}\{y \notin \arg \max(z)\}$$

This is not differentiable.

Error margin

If z is the positive class output then

$$\ell(z, y) = -\mathbb{I}\{zy < 0\} zy$$

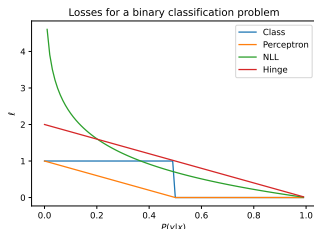
Used in the perceptron.

Negative log likelihood

If z are label probabilities, then

$$\ell(z, y) = -\ln z_y.$$

Used in logistic regression.



Hinge loss

If z are the output for each class

$$\ell(z, y) = 1 - z_y$$

Used in large margin classifiers.

Regression cost functions

L2 loss (Squared error)

If z is a prediction for y then

$$\ell(z, y) = (y - z)^2$$

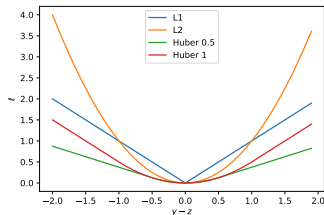
This is equivalent to negative log likelihood under Gaussianity. Used in linear regression.

L1 loss

If z is a prediction for y then

$$\ell(z, y) = |y - z|$$

Used in LASSO regression.



Huber loss

If z is a prediction, then

$$\ell(z, y) = \begin{cases} \frac{1}{2}(z - y)^2 & |z - y| \leq \delta \\ \delta(|z - y| - \frac{1}{2}\delta) & \text{otherwise.} \end{cases} \quad (6)$$

Mixes L1 and L2 losses.

Gradient descent in practice

The ideal gradient descent algorithm:

If we could calculate $\nabla_{\beta} \mathbb{E}_{\beta}[L]$, we could do:

$$\beta_{n+1} = \beta_n - \alpha_n \nabla_{\beta} \mathbb{E}_{\beta}[L]$$

for a suitable α_n schedule.

Gradient descent on the empirical error

Since we only have the data, we can try to minimise the empirical loss $\frac{1}{T} \sum_{t=1}^T \ell(x_t, y_t, \beta)$ through gradient descent

$$\beta_{n+1} = \beta_n - \alpha_n \frac{1}{T} \sum_{t=1}^T \nabla_{\beta} \ell(x_t, y_t, \beta)$$

This is also called **batch** gradient descent.

Stochastic gradient descent

Gradient descent on one example:

We don't have to wait calculate $\nabla_{\beta} \ell(x_t, y_t, \beta)$ for all t before applying the update. We can do it at every example:

$$\beta_{n+1} = \beta_n - \alpha_n \nabla_{\beta} \ell(x_{[n]_T}, y_{[n]_T}, \beta).$$

Here $[n]_T$ is $1 + n$ modulo T to ensure $n \in \{1, \dots, T\}$.

Minibatch gradient descent

However, it is a bit better to look at K examples at a time before we change the parameters. This is called a **minibatch**

$$\beta_{n+1} = \beta_n - \alpha_n \frac{1}{K} \sum_{k=nK}^{(n+1)K-1} \nabla_{\beta} \ell(x_{[k]_T}, y_{[k]_T}, \beta)$$

This also helps with parallelisation, since we can compute $\ell, \nabla_{\beta} \ell$ in parallel for each example.

Features and layers

Introduction

Layers

Activation functions

Algorithms

Random projection

Back propagation

Derivatives

Cost functions

Stochastic gradient descent in practice

Python libraries

sklearn

PyTorch

TensorFlow

sklearn neural networks

Classification

Uses the **cross entropy** cost

```
from sklearn.neural_network import MLPClassifier
clf = MLPClassifier(hidden_layer_sizes=(5, 2))
clf.fit(X, y)
clf.predict(X_test)
```

- Main condition is layer sizes.

Regression

```
from sklearn.neural_network import MLPRegressor
model = MLPRegressor(hidden_layer_sizes=(5, 2))
```

PyTorch

Data set-up

```
X_train = torch.tensor(X_train, dtype=torch.float32)
train_dataset = TensorDataset(X_train, y_train)
train_loader = DataLoader(train_dataset, batch_size=16,
```

PyTorch: Manual training

Network setup

```
fc1 = nn.Linear(input_size , hidden_size)
# Input to hidden layer
fc2 = nn.Linear(hidden_size , output_size)
# Hidden layer to output layer
sigmoid = nn.Sigmoid() # some activation function
criterion = nn.BCELoss() #what loss to minimise
optimizer = optim.SGD(model.parameters(), lr=0.001) # ho
```

Training

```
# Manual forward pass.
z1 = fc1(inputs) # hidden layer 1
a1 = sigmoid(z1) # Apply activation for hidden
z2 = fc2(a1) # Linear combination in output layer
outputs = sigmoid(z2) # Output layer activation
loss = criterion(outputs , labels) # Specify loss
loss.backward() # Backward pass
optimizer.step() # Update weights
```

TensorFlow

This is another library, no need to use this for this course